

ROGDI Reproduction Guide

This is the file to read when you want to understand, step by step, how the ROGDI protocol was made.

Start With This File

Start by reading:

`docs/ROGDI-deep-dive.md`

That is the best first file because it explains the actual science behind the ROGDI case study. It tells you:

- what disease is being studied
- what gene is involved
- what sources were checked
- why ROGDI might fit gene therapy
- what the biggest problems are
- why the final protocol says what it says

After that, read these files in this order:

1. `reports/ROGDI_Standardised_Gene_Therapy_Protocol.md` This is the final report that was generated.
2. `data/rogdi_test_fixture.json` This is the little box of checked facts about ROGDI. It stores the important true details in a structured way.
3. `src/nanogt/cli.py` This is where the program loads the ROGDI facts when you run the command.
4. `src/nanogt/report.py` This is where the program turns those facts into paragraphs, scores, risks, and recommendations.
5. `src/nanogt/templates/protocol.md.j2` This is the skeleton of the final report. It says where each fact and paragraph should appear.
6. `tests/test_nanogt.py` This checks that the program is using the corrected ROGDI facts and not old wrong ones.

The Big Picture

The AI did not magically create the ROGDI file in one jump.

It did this:

1. Pick one disease.
2. Check the disease identity.
3. Check the gene identity.
4. Check the protein identity.
5. Work out whether gene therapy makes sense.
6. Store the checked facts.
7. Put those facts into the code.
8. Use a report template.
9. Run a command.
10. Test that the report did not contain old mistakes.

Think of it like baking a cake:

- `docs/ROGDI-deep-dive.md` is the recipe notes.

- `data/rogdi_test_fixture.json` is the ingredient list.
- `src/nanogt/cli.py` chooses the ingredients.
- `src/nanogt/report.py` mixes them.
- `src/nanogt/templates/protocol.md.j2` is the cake tin shape.
- `reports/ROGDI_Standardised_Gene_Therapy_Protocol.md` is the finished cake.

Exact Steps Used To Create The ROGDI Protocol

Step 1: Choose The Disease

The disease chosen was:

Kohlschutter-Tonz syndrome / amelorcerbrohypohidrotic syndrome

The key disease identifier was:

ORPHA:1946

This matters because disease names can be messy. One disease can have more than one name. The identifier is the stable label.

What you should write down:

- disease name
- Orphanet ID
- OMIM phenotype ID
- inheritance pattern
- main symptoms
- whether there is already a treatment

For ROGDI, the checked values were:

- Orphanet: ORPHA:1946
- OMIM phenotype: 226750
- inheritance: autosomal recessive
- main symptoms: enamel defects, early epilepsy, severe neurodevelopmental impairment
- treatment: no disease-modifying therapy

Step 2: Find The Correct Gene

The disease was linked to the gene:

ROGDI

The AI had to be careful here because a wrong gene or protein identity would ruin the whole report.

For ROGDI, the checked values were:

- gene symbol: ROGDI
- OMIM gene ID: 614574
- chromosome: 16p12.1
- aliases: KIAA0267, FLJ22386, RAV2

Where this appears in the project:

- `docs/ROGDI-deep-dive.md`
- `data/rogdi_test_fixture.json`
- `src/nanogt/cli.py`

Step 3: Find The Correct Protein

The correct protein was:

Protein rogdi homolog

The correct UniProt ID was:

Q9GZN7

The important protein facts were:

- UniProt: Q9GZN7
- protein length: 287 aa
- molecular weight: 32,254 Da
- coding sequence length: about 861 bp
- not secreted
- intracellular

This step matters a lot.

If the protein is small, it may fit inside AAV.

If the protein is secreted, nearby cells might help each other.

If the protein is not secreted, each affected cell probably needs to receive the therapy itself.

ROGDI is not secreted, so the report says direct cell transduction is important.

Step 4: Check Sources Before Making Claims

The AI used source checking before writing the final scientific claims.

The main source types were:

- Orphanet for disease identity
- OMIM for disease and gene identity
- UniProt for protein identity
- PubMed papers for disease mechanism
- PDB / AlphaFold for structure
- Human Protein Atlas for expression
- FDA / precedent therapy sources for gene therapy comparison

For ROGDI, the important links are listed near the top of:

docs/ROGDI-deep-dive.md

The simple rule is:

Do not write a confident claim until you know where it came from.

Step 5: Ask If Gene Therapy Makes Sense

The AI then asked a set of practical questions.

Question 1:

Is this disease genetic?

For ROGDI: yes.

Question 2:

Is it recessive loss of function?

For ROGDI: yes, this supports gene addition.

Question 3:

Is the gene small enough for AAV?

For ROGDI: yes. The coding sequence is about 861 bp, much smaller than the rough AAV limit of 4700 bp.

Question 4:

What tissue needs treatment?

For ROGDI: mainly CNS neurons, but teeth are also affected.

Question 5:

Can one delivery route fix everything?

For ROGDI: probably not. Brain delivery and tooth/enamel delivery are different problems.

Question 6:

Is the protein secreted?

For ROGDI: no. This makes treatment harder because corrected cells probably cannot rescue nearby uncorrected cells.

Step 6: Decide The First Gene Therapy Strategy

The AI made a first-pass strategy.

For ROGDI, the chosen strategy was:

- vector: AAV9
- promoter: hSYN1
- cargo: human ROGDI cDNA
- route: CNS-prioritized systemic IV or intrathecal delivery
- first goal: CNS rescue
- dental repair: secondary or separate problem

In very simple words:

The first version tries to help the brain first, because epilepsy and neurodevelopmental problems are the most serious part.

It does not pretend that one brain-focused treatment will magically fix already-formed tooth enamel.

Step 7: Store The Checked Facts In JSON

The checked facts were written into:

`data/rogdi_test_fixture.json`

This file is like a tidy fact sheet that the tests can read.

It includes things like:

- gene name
- disease name
- UniProt ID
- Orphanet ID
- OMIM IDs
- protein length
- protein function hints
- symptoms
- AAV compatibility
- risks
- source links

This file helps stop the project from accidentally drifting back to old wrong facts.

Step 8: Put The ROGDI Facts Into The CLI

The code file:

`src/nanogt/cli.py`

contains a function called:

`_load_rogdi_data()`

That function creates the ROGDI objects used by the report:

- `Disease`
- `Gene`
- `Protein`
- `Vector`
- `ScoreBreakdown`

In plain English, this function says:

When someone asks for `ORPHA:1946`, load the corrected ROGDI case study.

The command checks this part:

`nanogt match --disease ORPHA:1946 --deep-dive --output reports/ROGDI_Standardised_Gene_Therapy_Pr`

Step 9: Write The Report Paragraphs In The Renderer

The file:

`src/nanogt/report.py`

does the heavy writing work.

It contains paragraphs for:

- disease description
- unmet need
- clinical course
- protein function
- localization
- expression pattern
- therapeutic rationale
- vector recommendation

- preclinical milestones
- regulatory pathway
- risks
- scoring
- go / no-go criteria
- final conclusion

The important function is:

```
render_protocol()
```

That function collects everything and sends it into the template.

Step 10: Use The Markdown Template

The file:

```
src/nanogt/templates/protocol.md.j2
```

is the report skeleton.

It has headings like:

- Executive Summary
- Indication Summary
- Target Biology
- Therapeutic Rationale
- Vector Assessment
- Preclinical Strategy
- Regulatory Pathway
- Risk Assessment
- Scoring Breakdown
- Go / No-Go Decision
- Conclusions

It also has placeholders like:

```
{{ gene.symbol }}
```

That means:

Put the gene symbol here.

For ROGDI, that becomes:

```
ROGDI
```

Another example:

```
{{ disease.name }}
```

That becomes:

```
Kohlschutter-Tonz syndrome / amelocerebrohypohidrotic syndrome
```

Step 11: Run The Command

The command used to generate the ROGDI protocol is:

```
nanogt match --disease ORPHA:1946 --deep-dive --output reports/ROGDI_Standardised_Gene_Therapy_Pr
```

If the installed command does not work, use:

```
uv run nanogt match --disease ORPHA:1946 --deep-dive --output reports/ROGDI_Standardised_Gene_The
```

Or, if you are running through Python directly:

```
uv run python -m nanogt.cli match --disease ORPHA:1946 --deep-dive --output reports/ROGDI_Standar
```

The command means:

- **nanogt**: run the project tool
- **match**: use the matching command
- **--disease ORPHA:1946**: use the ROGDI/KTS case
- **--deep-dive**: make the full protocol, not a tiny summary
- **--output ...**: save the file here

Step 12: Open The Finished File

The output file is:

```
reports/ROGDI_Standardised_Gene_Therapy_Protocol.md
```

That file is the final generated protocol.

It is not where the logic lives.

It is the final result.

If you want to change the final result permanently, change the input facts, renderer text, or template, then regenerate the report.

Step 13: Run Tests

The test file is:

```
tests/test_nanogt.py
```

The tests check things like:

- ROGDI is still ROGDI
- UniProt is still Q9GZN7
- protein length is still 287 aa
- coding sequence is still about 861 bp
- old wrong terms are not present
- the protocol contains the corrected ROGDI science

Run:

```
uv run pytest
```

If the tests pass, the project is still internally consistent.

Tiny Version Of The Workflow

If you only remember one thing, remember this:

1. Read `docs/ROGDI-deep-dive.md`.
2. Check the facts in `data/rogdi_test_fixture.json`.
3. See how `src/nanogt/cli.py` loads ROGDI.

4. See how `src/nanogt/report.py` writes the content.
5. See how `src/nanogt/templates/protocol.md.j2` shapes the report.
6. Run the `nanogt match` command.
7. Read the output in `reports/ROGDI_Standardised_Gene_Therapy_Protocol.md`.
8. Run `uv run pytest`.

How To Repeat This For A New Disease

Use this checklist.

A. Make A New Disease Audit

Create a new deep-dive note in `docs/`.

Example:

`docs/MYGENE-deep-dive.md`

Write:

- disease name
- gene name
- Orphanet ID
- OMIM disease ID
- OMIM gene ID
- UniProt ID
- protein length
- coding sequence length
- inheritance
- symptoms
- target tissues
- source links
- gene therapy pros
- gene therapy problems

B. Make A New Fixture

Create a new JSON file in `data/`.

Example:

`data/mygene_test_fixture.json`

Put the checked facts there.

C. Add A Loader In The CLI

In `src/nanogt/cli.py`, add a new function like:

`_load_mygene_data()`

That function should build the same five objects:

- Disease
- Gene
- Protein

- Vector
- ScoreBreakdown

D. Connect The Disease ID To The Loader

Still in `src/nanogt/cli.py`, update the `match()` command.

It currently has a special path for:

`ORPHA:1946`

For a new disease, add another path for the new Orphanet ID.

E. Update The Renderer If Needed

If the report needs new paragraphs, edit:

`src/nanogt/report.py`

If the report needs new sections or a different shape, edit:

`src/nanogt/templates/protocol.md.j2`

F. Add Tests

In `tests/test_nanogt.py`, add tests that check:

- the gene ID is correct
- the UniProt ID is correct
- the disease ID is correct
- the protein size is correct
- old wrong names are not present
- the generated report contains the important biology

G. Generate The New Report

Run:

`uv run nanogt match --disease YOUR_ORPHANET_ID --deep-dive --output reports/YOUR_REPORT_NAME.md`

H. Read The Report Like A Reviewer

Ask:

- Are the IDs correct?
- Are the sources real?
- Is the gene therapy logic honest?
- Does the report explain the hard parts?
- Does it avoid pretending uncertain things are certain?
- Do the tests pass?

If yes, the new disease report is ready as a first project draft.